

HALc: Heuristic Agent Ledger by Confrontation

A Distributed Coordination Protocol for Heterogeneous Large Language Model Agents

Table of Contents

[Key Takeaways](#)

[Executive Summary](#)

[Abstract](#)

[Positioning Statement](#)

[1. Introduction](#)

[2. Related Work](#)

[2.1 Constitutional Reasoning](#)

[2.2 Collaborative Multi-Agent Reasoning](#)

[2.3 Pairwise Ranking of LLM Outputs](#)

[2.4 Critique-Guided Improvement](#)

[2.5 Enterprise AI Coordination](#)

[2.6 Constructive Confrontation in Software Engineering](#)

[2.7 Social Choice Theory and Rank Aggregation](#)

[2.8 Persistent and Shared Agent Memory](#)

[2.9 Positioning of HALc](#)

[3. The HALc Model](#)

[3.1 Agent Population](#)

[3.2 Shared Semantic Context](#)

[3.3 User Request](#)

[3.4 Independent Solution Synthesis](#)

[3.5 Deterministic Pairwise Assignment](#)

[3.6 Pairwise Constructive Confrontation](#)

[3.7 Comparison Graph](#)

[3.8 Weighted Copeland Aggregation](#)

[3.9 Shared Knowledge Evolution](#)

[3.10 Computational Complexity](#)

[4. Mathematical Analysis](#)

[4.1 Comparison Graph](#)

[4.2 Core Theorems](#)

[4.3 Weighted Copeland Ranking](#)

[4.4 Structural Invariants](#)

[4.5 Computational Complexity](#)

[4.6 Summary of Formal Guarantees](#)

[4.7 Discussion - The Approximation Gap](#)

[5. Implementation](#)

[5.1 Architectural Overview](#)

[5.2 Technology Stack](#)

[5.3 Agent Configuration](#)

[5.4 OpenAI-Compatible Interface](#)

[5.5 Orchestration Engine](#)

[5.6 Markdown Communication Protocol](#)

[5.7 Pairwise Assignment](#)

[5.8 Weighted Copeland Aggregation](#)

[5.9 Shared Knowledge Ledger](#)

[5.10 Observability and Traceability](#)

[5.11 Error Handling](#)

[5.12 Testing Strategy](#)

[5.13 Extensibility](#)

[5.14 Experimental Design](#)

[5.14.1 Research Questions](#)

[5.14.2 Baselines](#)

[5.14.3 Benchmark Tasks](#)

[5.14.4 Metrics](#)

[5.14.5 Ledger Accumulation Protocol](#)

[5.14.6 Agent Population](#)

[6. Discussion](#)

[6.1 Coordination Rather than Debate](#)

[6.2 Separation of Reasoning and Coordination](#)

[6.3 Markdown as a Semantic Exchange Format](#)

[6.4 The Shared Knowledge Ledger as Primary Contribution](#)

[6.5 Continuous Organizational Learning](#)

[6.6 Confidence-Weighted Decision Making](#)

[6.7 Scalability Considerations](#)

[6.8 Applicability to Enterprise AI](#)

[6.9 Trade-offs](#)

[6.10 Relationship to Existing Work](#)

[6.11 Summary](#)

[7. Limitations](#)

[7.1 Absence of Empirical Evaluation](#)

[7.2 Dependence on LLM Quality](#)

[7.3 Confidence Calibration](#)

[7.4 Sparse Comparison Topology](#)

[7.5 Static Reviewer Assignment](#)

[7.6 Shared Context Growth](#)

[7.7 Single Shared Memory](#)

[7.8 Robustness to Faulty or Adversarial Agents](#)

[7.9 Determinism vs. Exploration](#)

[7.10 Generality of the Coordination Protocol](#)

[7.11 Absence of Formal Optimality Proof](#)

[7.12 Summary](#)

[References](#)

[Multi-Agent Reasoning and Constitutional AI](#)

[Enterprise AI Coordination](#)

[Software Architecture and Constructive Confrontation](#)

[Distributed Systems](#)

[Social Choice Theory](#)

[Decision Theory and Collective Intelligence](#)

[Large Language Models](#)

[Mathematical Foundations](#)

[HALc](#)

Key Takeaways

HALc's most distinctive contribution is the shared versioned knowledge ledger, a mechanism that allows heterogeneous LLM agents to accumulate collective organizational knowledge across successive reasoning sessions, rather than discarding it or fragmenting it across isolated agent memories.

While existing research has addressed communication efficiency in multi-agent debate, no comparable mechanism appears in the multi-agent coordination literature reviewed here for persistent, auditable, evolving collective memory in enterprise AI deployments. HALc introduces this through a versioned Markdown ledger that captures winning proposals, lessons learned, remaining risks, and unresolved disagreements after every confrontation.

This ledger is supported by two enabling architectural choices: a deterministic ring-based confrontation protocol that reduces communication complexity from $O(N^2)$ to $O(N)$, and a Markdown-native communication layer that ensures provider-independent interoperability across heterogeneous LLM agents.

Together, these design choices shift the focus of enterprise multi-agent AI from isolated reasoning sessions toward communities of agents that accumulate and reuse shared organizational knowledge across successive reasoning sessions.

Executive Summary

Large Language Models (LLMs) have demonstrated remarkable capabilities in reasoning, software engineering, and decision support. However, current enterprise deployments increasingly reveal that the primary limitation is no longer the reasoning capability of individual models, but the coordination of multiple specialized agents. Existing multi-agent systems frequently rely on either centralized orchestration, sequential critique loops, or exhaustive all-to-all debate, leading to high computational costs, redundant reasoning, limited scalability, and complex operational behavior.

This paper introduces **HALc (Heuristic Agent Ledger by confrontation)**, a distributed multi-agent reasoning protocol that applies principles of constructive confrontation, constitutional guidance, and social choice theory to produce robust collective decisions with linear communication complexity. HALc treats each agent as an autonomous expert operating under a shared semantic context while maintaining distinct reasoning strategies defined by individual prompts and models. Rather than requiring every agent to evaluate every other proposal, HALc organizes the population into a deterministic pairwise comparison graph. Each agent produces a complete solution synthesis and performs exactly one comparison between two peer proposals. The resulting directed weighted comparison graph is aggregated using a **Weighted Copeland ranking**, producing a globally ordered set of alternatives while requiring only **$2N$** LLM interactions for a population of **N** agents, compared with the **$O(N^2)$** communication cost of traditional debate-based approaches.

Unlike existing debate frameworks that optimize for consensus through repeated exchanges, HALc emphasizes **constructive confrontation**. Agents are encouraged to independently develop solutions, evaluate competing proposals according to shared organizational priorities, and express confidence-weighted preferences. The protocol deliberately separates reasoning from coordination: agents communicate exclusively through structured Markdown documents, while HALc performs deterministic parsing, ranking, orchestration, and ledger management. This design enables heterogeneous OpenAI-compatible models to participate without requiring identical output schemas or provider-specific capabilities.

HALc further introduces the concept of a **shared evolving semantic context**. Instead of maintaining isolated memories for individual agents, every confrontation produces a new versioned Markdown knowledge ledger that captures the winning proposal, collective lessons learned, remaining risks, and unresolved disagreements. This shared context becomes the common baseline for subsequent reasoning rounds, allowing the agent population to accumulate organizational knowledge while avoiding fragmentation of individual memories.

The proposed protocol synthesizes concepts originating from Constitutional AI, critique-guided self-improvement, collaborative reasoning, software architecture review methodologies, distributed systems, and social choice theory into a unified enterprise-oriented coordination framework. We demonstrate that deterministic pairwise confrontation preserves sufficient comparison information to construct a stable global ranking while dramatically reducing computational cost. The resulting architecture offers a practical balance between reasoning diversity, explainability, scalability, and operational efficiency, making it suitable for enterprise AI systems where coordination, governance, and reproducibility are increasingly more important than maximizing the capabilities of any individual language model.

HALc therefore shifts the focus of multi-agent AI from *larger models reasoning longer* toward *communities of specialized agents reasoning together under structured confrontation*, providing a new protocol for scalable, explainable, and continuously improving collective intelligence.

Abstract

As enterprise AI evolves from single-model systems to communities of specialized agents, the primary challenge shifts from reasoning capability to coordination. Existing multi-agent approaches rely on centralized orchestration or exhaustive all-to-all debate, resulting in quadratic communication costs, limited scalability, and brittle provider-specific interfaces. Recent research has addressed communication efficiency through learned sparse topologies and pairwise ranking mechanisms. However, these contributions remain largely disconnected from the operational requirements of enterprise deployments, where governance, reproducibility, provider independence, and organizational memory are primary constraints.

This paper introduces **HALc (Heuristic Agent Ledger by Confrontation)**, a distributed coordination protocol that synthesizes sparse confrontation, constitutional guidance, and Weighted Copeland ranking into a unified, deployable system for heterogeneous LLM agents. HALc makes three architectural contributions absent from prior work taken individually.

First, a deterministic ring-based pairwise confrontation algorithm requiring exactly $2N$ LLM interactions for N agents, designed for reproducibility and auditability rather than adaptivity. Second, a Markdown-native communication protocol that enables interoperability across heterogeneous OpenAI-compatible providers without brittle structured output requirements. Third, a shared versioned knowledge ledger that evolves after each confrontation, enabling collective organizational learning across successive executions.

Together, these design choices provide a practical coordination architecture for enterprise multi-agent systems where traceability, explainability, and operational simplicity are as important as reasoning quality.

Positioning Statement

As mentioned earlier, the core mechanisms underlying HALc, sparse pairwise communication, LLM output ranking by confrontation, and social choice aggregation, have prior and concurrent treatments in the research literature. Chen et al. (2024) address the same transition from fully-connected multi-agent debate to sparse communication graphs, demonstrating quality preservation at reduced cost. Jiang et al. (2023) establish pairwise ranking of heterogeneous LLM outputs as a principled ensembling method, predating HALc's confrontation mechanism. The use of Copeland's method to aggregate pairwise LLM preferences into global rankings is documented in Lanctot et al. (2025) and Kaesberg et al. (2025), among others. The quadratic communication cost of exhaustive debate is a well-recognized limitation addressed by a growing body of work including AgentPrune (Zhang et al., 2025a), AgentDropout (Wang et al., 2025b), G-Designer (Zhang et al., 2025b), and GoAgent (Tran et al., 2026), which go beyond fixed topologies to learn task-adaptive sparse structures. Constitutional guidance through explicit principle documents originates with Bai et al. (2022).

HALc does not claim to introduce these ideas independently. Rather, it integrates them into a unified, deployable coordination protocol designed for heterogeneous enterprise LLM environments. Its primary contributions are architectural: a deterministic ring-based confrontation topology that prioritizes reproducibility and auditability over adaptivity; a Markdown-native communication protocol that eliminates provider-specific structured output requirements; and a shared versioned knowledge ledger that enables collective organizational learning across successive confrontations.

These design choices reflect enterprise constraints - governance, traceability, operational simplicity - that are largely absent from the academic literature cited above. HALc should therefore be understood as a systems contribution that synthesizes existing research into a coherent, implementable protocol, rather than as a proposal of novel theoretical primitives.

Research Area	Prevailing Industry Approach (The Gap)	HALc Positioning (The Contribution)
Communication Complexity	Quadratic ($O(N^2)$): Exhaustive debate/all-to-all critique models.	Linear ($O(N)$): Deterministic pairwise comparison graph with exactly one review per agent.
Reasoning Method	Iterative Refinement: Multiple rounds of refinement on a single proposal.	Constructive Confrontation: One-shot solution synthesis + weighted pairwise ranking.
Orchestration	Centralized/Black-box: Implicit orchestration via rigid JSON schemas.	Decoupled/Deterministic: Explicit orchestration, Markdown-native, and agnostic of provider-specific output formats.
Memory/Context	Fragmented: Isolated agent memories or simple prompt histories.	Shared Evolving Ledger: Versioned Markdown context capturing collective knowledge/history.
Aggregation	Majority Voting: Often simplistic or opaque model-based selection.	Formal Social Choice: Weighted Copeland ranking for stable, reproducible global orders.

1. Introduction

Large Language Models have rapidly evolved from conversational assistants into autonomous software components capable of planning, coding, reasoning, and interacting with external systems. Recent advances have shifted research attention from improving individual models toward orchestrating multiple specialized agents that collaborate to solve increasingly complex tasks. This evolution reflects a broader transition in enterprise AI: the limiting factor is no longer the intelligence of a single model, but the ability of many models to coordinate effectively, and over time, to learn collectively.

Current multi-agent systems typically address the first part of this challenge, coordination, through one of three patterns: centralized orchestration, iterative self-critique, or multi-agent debate. Each has demonstrated improvements in reasoning quality, but all share a deeper limitation that the research community has largely left unaddressed. When a session ends, collective knowledge disappears. Individual agents may retain isolated memories, or context may be discarded entirely. In either case, the organization that deployed those agents gains nothing from the accumulated reasoning of previous sessions. Every new request starts from scratch.

This is the gap HALc addresses.

The research community has made significant progress on communication efficiency. Chen et al. (2024) demonstrated that sparse topologies preserve reasoning quality while reducing interaction overhead. AgentPrune, AgentDropout, and G-Designer introduced learned sparse graphs that adapt to task requirements. On the aggregation side, Jiang et al. (2023) established pairwise ranking as an effective ensembling strategy, while Lanctot et al. (2025) and Kaesberg et al. (2025) demonstrated the applicability of Condorcet-style social choice methods to multi-agent LLM settings. HALc draws on all of these contributions. But it asks a different question: once agents have reasoned together, what happens to what they learned?

Enterprise deployments differ from laboratory multi-agent systems in several important ways. Organizations deploy heterogeneous ecosystems of models from multiple providers, specialized across domains such as architecture, cybersecurity, compliance, and product management. These environments impose requirements that research frameworks rarely address: full execution traceability, deterministic reproducible decisions, provider-independent interoperability, and the accumulation of organizational knowledge across successive reasoning sessions.

This paper introduces HALc (Heuristic Agent Ledger by Confrontation), a distributed coordination protocol designed specifically for these constraints. Its primary contribution is the shared versioned knowledge ledger. After every confrontation, HALc generates a new versioned Markdown document capturing the user request, winning proposal, complete ranking, lessons learned, remaining risks, and unresolved disagreements. This document becomes the shared constitutional context for all agents in the next execution. Organizational knowledge therefore

evolves collectively rather than individually, providing both a persistent audit trail and a continuously improving foundation for future reasoning sessions. While related work has explored persistent and shared agent memory, HALc introduces a specific integration of confrontation-based ranking with a versioned, human-readable decision ledger. The ledger records winning proposals, rankings, risks, lessons learned, and unresolved disagreements, then reuses this information as shared context for subsequent executions.

This ledger is made practical by two supporting architectural contributions. First, a deterministic ring-based pairwise confrontation algorithm that reduces communication complexity from $O(N^2)$ to exactly $2N$ LLM interactions for N agents, prioritizing reproducibility and operational auditability over adaptive topology optimization. Second, a Markdown-native communication protocol that enables provider-independent interoperability across heterogeneous OpenAI-compatible endpoints without brittle structured output requirements.

The protocol strictly separates reasoning from coordination. Agents perform reasoning; HALc performs orchestration, parsing, ranking, ledger management, and execution tracing. This separation allows any OpenAI-compatible endpoint to participate without modifying the coordination algorithm, and ensures identical outputs given identical inputs.

The main contributions of this work are:

1. **A shared versioned knowledge ledger** that accumulates collective organizational learning across successive confrontations, providing both a persistent audit trail and an evolving constitutional context for future reasoning sessions - the primary and most distinctive contribution of HALc.
2. A deployable distributed coordination protocol for heterogeneous LLM agents **that** synthesizes sparse confrontation, constitutional guidance, and Weighted Copeland ranking into a unified enterprise-oriented system.
3. **A deterministic ring-based pairwise confrontation algorithm** requiring exactly $2N$ LLM interactions for N agents, prioritizing reproducibility and operational auditability over adaptive topology optimization.
4. **A Markdown-native communication protocol** enabling provider-independent interoperability across heterogeneous OpenAI-compatible endpoints without brittle structured output requirements.
5. **An implementation in Rust exposing an OpenAI-compatible API**, validating the practical feasibility of the protocol: <https://github.com/gegout/HALc>

The remainder of this paper first reviews related work on constitutional reasoning, collaborative agents, constructive critique, and social choice theory. It then formalizes the HALc protocol, analyzes its computational properties, discusses implementation considerations, and outlines an experimental methodology for evaluating coordination efficiency, reasoning quality, scalability, and robustness in enterprise multi-agent systems.

2. Related Work

The design of HALc draws upon several complementary research areas that have evolved largely independently over the last decade. These include constitutional reasoning, collaborative multi-agent systems, constructive critique and self-improvement, software architecture review methodologies, and social choice theory. While each field addresses a specific aspect of collective reasoning, few provide a complete coordination protocol suitable for heterogeneous enterprise AI systems. HALc proposes such a protocol by combining contributions from these domains into a unified framework. This section situates HALc within the existing literature and clarifies where it synthesizes prior contributions versus where it introduces original design choices.

2.1 Constitutional Reasoning

One of the major challenges in multi-agent systems is ensuring that autonomous agents reason consistently while preserving their independence. Anthropic introduced **Constitutional AI**, in which language models critique and revise their own responses according to an explicit set of principles rather than relying exclusively on human feedback. This approach demonstrated that structured self-critique can substantially improve harmlessness and alignment while reducing dependence on reinforcement learning from human feedback.

HALc adopts the fundamental principle that reasoning should be guided by an explicit constitution. However, instead of defining the constitution as ethical rules, HALc generalizes the concept to enterprise decision making. The shared `context.md` becomes a semantic constitution describing organizational priorities, architectural principles, business constraints, evaluation criteria, and decision objectives. Every agent interprets competing proposals according to this common constitutional document, allowing heterogeneous models to evaluate alternatives using consistent priorities while preserving their individual expertise and reasoning strategies.

Unlike Constitutional AI, which focuses on improving the reasoning of a single model through iterative self-critique, this application of constitutional guidance to collective agent coordination is a reframing of Bai et al. (2022) rather than an independent contribution. HALc extends constitutional guidance to a distributed population of independent agents participating in collective decision making.

2.2 Collaborative Multi-Agent Reasoning

Recent research has demonstrated that collaboration among multiple language models frequently produces higher-quality solutions than isolated reasoning. Multi-agent debate frameworks encourage independent proposal generation followed by iterative critique among participating agents. Similarly, Meta's **Collaborative Reasoner (CoRAL)** demonstrates that synthetic collaborative conversations enable agent populations to improve reasoning through social interaction rather than isolated inference.

The quadratic communication cost of exhaustive all-to-all debate is a well-recognized limitation in the literature. Chen et al. (2024) directly address this in their AAAI 2025 workshop contribution, demonstrating that sparse communication topologies preserve reasoning quality while substantially reducing interaction overhead. Subsequent work has extended this direction significantly. AgentPrune (Zhang et al., 2025a) and AgentDropout (Wang et al., 2025b) learn task-specific sparse graphs by eliminating suboptimal connections and redundant agents. G-Designer (Zhang et al., 2025b) introduces a graph sparsification learning objective for communication design. GoAgent (Tran et al., 2026) and ARG-Designer (Li et al., 2025) go further still, generating communication topologies dynamically from scratch using autoregressive and diffusion-based models.

HALc operates in the same problem space but adopts a deliberately different design philosophy. Rather than learning or adapting the communication topology, HALc uses a fixed deterministic ring assignment that guarantees balanced reviewer workloads, connected comparison coverage, and fully reproducible execution given identical inputs. This choice sacrifices the adaptivity of learned topologies in favor of auditability, operational simplicity, and deterministic behavior - properties that are particularly important in enterprise governance contexts. HALc should therefore be understood as occupying a distinct point in the design space: not superior to adaptive approaches in general reasoning quality, but preferable where reproducibility and traceability are primary requirements.

2.3 Pairwise Ranking of LLM Outputs

The use of pairwise comparison to rank outputs from multiple language models was established by Jiang et al. (2023) in LLM-Blender, an ensembling framework comprising two modules: PairRanker, which uses cross-attention encoders to distinguish between candidate outputs through pairwise comparison, and GenFuser, which merges top-ranked candidates into a synthesized response. LLM-Blender demonstrated that pairwise ranking of heterogeneous LLM outputs significantly outperforms individual models and simple aggregation baselines.

HALc's confrontation mechanism draws on the same foundational insight: that pairwise comparison of candidate solutions is an effective and principled method for identifying higher-quality outputs across a heterogeneous model population. The key distinction is architectural. LLM-Blender employs a trained cross-encoder model as the ranking component, operating outside the agent population. HALc instead delegates comparison to participating LLM agents themselves, which evaluate competing proposals according to a shared constitutional context and express confidence-weighted preferences in natural language. This design preserves the reasoning transparency of the confrontation process and avoids the need for a separately trained ranking model, at the cost of relying on the calibration quality of the reviewer agents.

2.4 Critique-Guided Improvement

Several studies investigate the role of structured criticism in improving LLM performance. Madaan et al. (2024) demonstrate iterative self-refinement through self-feedback, while Shinn et al. (2024) introduce verbal reinforcement learning through reflective critique. These approaches consistently improve solution quality in software engineering, planning, and mathematical reasoning tasks.

HALc shares the philosophy that criticism is essential for high-quality reasoning but differs in two important respects. First, critique is directed toward competing proposals rather than toward an agent's own reasoning. Second, confrontation occurs exactly once per agent, producing confidence-weighted pairwise preferences rather than multiple refinement iterations. Criticism therefore becomes a mechanism for comparing alternative solutions rather than repeatedly editing the same output, significantly reducing the number of required LLM interactions.

2.5 Enterprise AI Coordination

As organizations increasingly deploy multiple specialized AI agents, several industrial analyses have argued that enterprise AI is becoming fundamentally a coordination problem rather than a model capability problem (Microsoft, 2025; TechTarget, 2025). Practical deployments require governance, explainability, reproducibility, interoperability across heterogeneous providers, and efficient orchestration of specialized models.

Existing orchestration frameworks primarily focus on workflow execution, task delegation, or tool invocation. They rarely define how conflicting recommendations from independent agents should be resolved or how organizational knowledge should evolve after collective decision making.

HALc addresses this gap through three key architectural principles.

- First, communication between agents is Markdown-native, allowing heterogeneous models to communicate naturally without rigid structured output schemas.
- Second, deterministic orchestration is separated from reasoning. HALc performs parsing, scheduling, ranking, persistence, and ledger management, while reasoning remains entirely inside participating agents.
- Third, every confrontation produces a versioned Markdown knowledge ledger that captures collective learning. This shared evolving context becomes the starting point for subsequent confrontations, allowing organizational knowledge to evolve continuously. This shared evolving context is the most distinctive contribution of HALc relative to prior work, and it is discussed in detail in Section 3.

2.6 Constructive Confrontation in Software Engineering

Long before the emergence of LLMs, software engineering recognized the importance of structured disagreement. The Rational Unified Process (Kruchten, 1995; Bass et al., 2022) formalized architecture reviews as collaborative exercises in which stakeholders challenge assumptions, expose risks, and improve architectural decisions through constructive criticism rather than consensus seeking.

HALc adopts this philosophy directly. Agents are not expected to reach immediate agreement; instead, disagreement is treated as a mechanism for improving collective decision quality. Every comparison explicitly identifies strengths, weaknesses, remaining risks, and confidence, transforming confrontation into a structured engineering process rather than an adversarial debate.

2.7 Social Choice Theory and Rank Aggregation

Once independent evaluations have been collected, the remaining challenge is aggregating potentially conflicting preferences into a single collective decision. This problem has been extensively studied within social choice theory and voting systems.

The Condorcet paradox demonstrates that pairwise preferences may produce cyclic rankings in which no globally preferred alternative exists. Numerous aggregation methods have been proposed to address this problem, including Borda Count, Kemeny-Young optimization, Ranked Pairs, the Schulze method, and Copeland ranking.

The application of these methods to LLM agent populations is an active area of research. Lanctot et al. (2025) and Kaesberg et al. (2025) apply Copeland's method, Ranked Pairs, and Iterative Maximal Lotteries to aggregate pairwise outcomes from LLM agents into global rankings, demonstrating that Condorcet-style methods provide more robust aggregation than majority voting in multi-agent settings.

HALc adopts a **Weighted Copeland** aggregation because it combines several desirable properties for distributed AI coordination. It operates directly on pairwise comparisons, naturally supports confidence-weighted preferences, remains computationally efficient, and produces an interpretable global ordering rather than only identifying a single winner. Confidence values provided by reviewer agents become edge weights within the comparison graph, enabling stronger preferences to contribute proportionally more to the final ranking while maintaining deterministic computation. The confidence-weighted variant is an incremental contribution relative to standard Copeland rather than a fundamental departure from prior aggregation methods.

2.8 Persistent and Shared Agent Memory

Several systems have addressed the challenge of maintaining memory across agent sessions. MemGPT (Packer et al., 2023) introduces a hierarchical memory architecture that pages information between the active context window and external storage, enabling agents to retain knowledge beyond their immediate context. A-MEM (Xu et al., 2025) organizes agent memories into self-evolving interconnected knowledge networks using structured attributes and semantic links. Zep (Rasmussen et al., 2025) builds a temporal knowledge graph that unifies and retrieves evolving conversational and business data across sessions. Also LangMem (<https://github.com/langchain-ai/langmem>) provides an SDK-level abstraction for extracting, storing, and retrieving persistent memories across agent interactions.

These systems demonstrate that persistent memory is a recognized requirement in agent deployments. However, they address memory from the perspective of individual agents or single-agent continuity. None introduces a mechanism for *collective* memory shared across a population of heterogeneous agents engaged in structured multi-agent confrontation. HALc's shared versioned ledger is therefore complementary to these approaches rather than redundant: it operates at the coordination layer rather than the individual agent layer, capturing collective organizational knowledge that no single agent accumulates independently.

2.9 Positioning of HALc

The core mechanisms underlying HALc - sparse pairwise communication, LLM output ranking by confrontation, and social choice aggregation - have prior and concurrent treatments in the research literature reviewed above. HALc does not claim to introduce these ideas independently. Rather, it integrates them into a unified, deployable coordination protocol designed specifically for heterogeneous enterprise LLM environments.

From Constitutional AI, HALc adopts explicit constitutional guidance through a shared semantic context.

From collaborative reasoning, it preserves independent proposal generation while dramatically reducing communication complexity.

From critique-guided improvement, it adopts structured criticism as the primary mechanism for evaluating alternatives.

From software architecture review methodologies, it adopts constructive confrontation as a disciplined decision-making process.

Finally, from social choice theory, it adopts Weighted Copeland ranking to transform sparse pairwise comparisons into a deterministic global ordering.

The resulting protocol shifts the emphasis of multi-agent AI away from maximizing interaction volume toward maximizing the information conveyed by each interaction. By reducing

communication complexity from quadratic to linear while preserving independent reasoning and explainable decision making, HALc provides a practical coordination architecture for heterogeneous populations of autonomous AI agents.

These design choices reflect enterprise constraints - governance, traceability, operational simplicity, and provider independence - that are still too absent from the academic multi-agent literature. HALc should therefore be understood as a systems contribution that synthesizes existing research into a coherent, implementable protocol, rather than as a proposal of novel theoretical primitives.

3. The HALc Model

This section formalizes HALc (Heuristic Agent Ledger by confrontation) as a distributed coordination protocol for heterogeneous Large Language Model agents. HALc separates reasoning from coordination, allowing agents to remain autonomous while producing deterministic collective decisions through structured confrontation.

3.1 Agent Population

Let the population of participating agents be:

$$A = \{a_0, a_1, \dots, a_{N-1}\} \text{ where } N \text{ is the number of active proposer agents.}$$

Each agent is configured as:

$$a_i = (E_i, M_i, P_i, K_i, \Theta_i)$$

where E_i is the endpoint, M_i the model, P_i the system prompt, K_i the credentials, and Θ_i the inference parameters.

3.2 Shared Semantic Context

The shared semantic context is denoted $C_{\square}$. After each confrontation, HALc generates a new shared context $C_{\square+1}$. The knowledge evolves as:

$$C_0 \rightarrow C_1 \rightarrow C_2 \rightarrow \dots$$

3.3 User Request

The user submits a request R . HALc broadcasts the tuple $(R, C_{\square})$ to every participating agent.

3.4 Independent Solution Synthesis

Each agent independently computes:

$$S_i = f_i(R, C_{\square}) \text{ where } f_i \text{ is the reasoning function of agent } a_i.$$

3.5 Deterministic Pairwise Assignment

For each agent a_i :

$$a_i \rightarrow (a_{i+1 \bmod N}, a_{i+2 \bmod N})$$

Each reviewer compares exactly two peer proposals, never its own. Every proposal is evaluated twice for $N \geq 3$, the comparison graph is connected, and communication complexity is linear.

3.6 Pairwise Constructive Confrontation

Each reviewer compares:

$$S_{\square} \text{ versus } S_{\square} \text{ with } i \neq j, i \neq k, \text{ and } j \neq k.$$

The reviewer returns a winner, a loser, and a confidence weight $w \in [0,10]$.

3.7 Comparison Graph

The comparisons form a weighted directed graph $G = (V, E)$, where $V = A$ and each edge (u, v, w) represents a victory of u over v with confidence w .

3.8 Weighted Copeland Aggregation

Define:

$$W_i^+ = \sum \text{wins weights}$$

$$W_i^- = \sum \text{loss weights}$$

$$\text{Score}(a_i) = W_i^+ - W_i^-$$

The final ranking is obtained by sorting agents by decreasing score.

3.9 Shared Knowledge Evolution

The shared context evolves according to:

$$C_{\square+1} = g(C_{\square}, R, L) \text{ where } L \text{ is the final ranking.}$$

3.10 Computational Complexity

HALc performs N synthesis calls, N pairwise comparison calls, and optionally one context generation call.

$$T(N) = 2N + \delta \text{ where } \delta = 1 \text{ if a summarizer agent is used, otherwise } 0.$$

Therefore $T(N) \in O(N)$, compared with $O(N^2)$ for exhaustive all-to-all debate. This result concerns the number of LLM invocations. Total token and network costs also depend on proposal length, shared-context size, retries, and provider behaviour. Assuming bounded proposal and context lengths, communication volume also scales linearly with (N) . Without this assumption, context growth may become the dominant cost.

HALc guarantees deterministic local coordination: identical proposals, comparison outcomes, weights, and configuration produce the same ranking. It does not guarantee identical end-to-end LLM outputs, since remote model inference may remain nondeterministic or change between provider versions.

The reduction in communication cost is achieved without sacrificing independent reasoning, structured confrontation, or deterministic collective decision making. This makes HALc suitable for large heterogeneous populations of autonomous enterprise AI agents.

4. Mathematical Analysis

The following analysis formalizes the mathematical and computational properties of the **Compact Distributed Pairwise Confrontation (CDPC)** protocol. We evaluate the comparison graph topology, the extent of evaluation coverage, the formal ranking mechanism, and the resulting communication complexity. The section concludes by consolidating structural invariants derived from the deterministic ring assignment into a unified proposition.

4.1 Comparison Graph

Let the ordered population of N proposer agents be defined as:

$$A = \{a_0, a_1, \dots, a_{N-1}\}, \text{ where } N \geq 3.$$

Every agent a_i synthesizes a unique proposal S_i and subsequently transitions to the reviewer role, conducting a structured pairwise comparison between the proposals generated by peer agents $a_{(i+1) \bmod N}$ and $a_{(i+2) \bmod N}$.

This distributed confrontation methodology yields a weighted directed comparison graph:

$$G = (V, E, w)$$

where $V = A$, $E \subseteq V \times V$, and $w : E \rightarrow [0, 10]$.

For each reviewer a_i , exactly one directed edge (u, v, w_i) is instantiated, representing the winning proposal u , the losing proposal v , and the confidence-weighted preference w_i as assigned by the reviewer.

4.2 Core Theorems

Theorem 1 - Communication Complexity

Statement. For a population comprising $N \geq 3$ agents, the CDPC protocol involves exactly N pairwise confrontations, resulting in exactly N edges within graph G .

Proof. Each agent a_i executes precisely one confrontation, thereby contributing a single edge to E . Given that $|A| = N$, the cumulative edge count is exactly N . ■

Remark. This demonstrates a significant reduction compared to exhaustive debate models, which incur a quadratic cost of $N(N-1)$. The comparative efficiency ratio is established as:

$$T_{\text{debate}(N)} / T_{\text{CDPC}(N)} = (N-1) / 2 \approx N/2 \text{ as } N \text{ increases.}$$

Theorem 2 - Evaluation Coverage

Statement. For $N \geq 3$, each proposal S_i is subjected to review by exactly two independent peer agents.

Proof. Agent a_i reviews the proposals $S_{(j+1) \bmod N}$ and $S_{(j+2) \bmod N}$. A specific proposal S_i is included in a_i 's assigned confrontation if and only if:

$$j + 1 \equiv i \pmod{N} \text{ or } j + 2 \equiv i \pmod{N}$$

Solving for the reviewer index yields $j \equiv i-1 \pmod{N}$ or $j \equiv i-2 \pmod{N}$. For $N \geq 3$, these indices remain distinct, as they would only coincide if $N \mid 1$, which is a contradiction. Consequently, exactly two peer agents - $a_{(i-1) \bmod N}$ and $a_{(i-2) \bmod N}$ - evaluate each proposal S_i . ■

Corollary. The total population-wide evaluation count is $2N$, maintaining linear scaling compared to the $N(N-1)$ cost of exhaustive approaches.

Theorem 3 - Topological Connectivity

Statement. For $N \geq 3$, the underlying undirected comparison graph G is characterized by full connectivity.

Proof. Consider the review sequence: a_0 evaluates S_1 and S_2 ; a_1 evaluates S_2 and S_3 ; and generally, a_i reviews $S_{(i+1) \bmod N}$ and $S_{(i+2) \bmod N}$

The proposals constitute a cyclic sequence $S_1, S_2, \dots, S_{N-1}, S_0$, representing a complete permutation. As every consecutive pair in this sequence is linked by a shared reviewer, a Hamiltonian cycle is induced across all nodes. Indeed, the sequence of reviews - comprising the pairs a_0 assessing S_1 and S_2 , a_1 assessing S_2 and S_3 , and continuing through to a_{N-1} evaluating S_0 and S_1 - constitutes a full permutation of proposal interactions.

This cyclic evaluation pattern induces a Hamiltonian cycle within the undirected comparison graph, thereby guaranteeing that every node remains transitionally connected.

Any graph containing such a cycle is inherently connected. ■

Proposition 1 — Ranking Approximation Bounds

Statement. Graph connectivity established in the previous **Theorem 3 (Topological Connectivity)** is necessary but not sufficient to guarantee that the Weighted Copeland ranking produced by CDPC matches the ranking that would result from exhaustive all-to-all pairwise evaluation.

Proof. We establish necessity and then demonstrate insufficiency by construction.

Necessity. A disconnected comparison graph contains at least two components with no edges between them. For any agent a_i in one component and a_j in another, no comparison has been performed between S_i and S_j , directly or transitively. The Weighted Copeland scores of a_i and a_j are therefore computed from disjoint edge sets with no shared information. The relative ordering of a_i and a_j in the final ranking is then determined entirely by tie-breaking rules rather than by comparison evidence. Connectivity is therefore a necessary condition for any ranking based on comparison evidence to be meaningful across the full agent population.

Insufficiency. Consider a population of $N=4$ agents $\{a_0, a_1, a_2, a_3\}$ under CDPC. The assignment produces four comparisons:

a_0 reviews S_1 vs S_2
 a_1 reviews S_2 vs S_3
 a_2 reviews S_3 vs S_0
 a_3 reviews S_0 vs S_1

This graph is connected by Theorem 4. However, the pair (S_0, S_2) is never directly compared, nor is the pair (S_1, S_3) . Their relative ordering in the final ranking is inferred transitively through shared edges rather than observed directly.

Now suppose the true preference order under exhaustive evaluation is $S_2 \succ S_0 \succ S_3 \succ S_1$, but S_0 receives high-confidence wins in its two observed comparisons while S_2 receives low-confidence wins. The Weighted Copeland score of S_0 may then exceed that of S_2 despite S_2 being preferred in exhaustive evaluation. The sparse ranking and the exhaustive ranking therefore diverge.

This divergence is not a protocol error, it is by design as it takes into account the confidence factor. It is a direct consequence of the information reduction inherent in sparse comparison. ■

4.3 Weighted Copeland Ranking

Each structured confrontation performed by reviewer a_i identifies a winning proposal u , a losing proposal v , and a confidence weight $w_i \in [0, 10]$, which is archived as a directed edge (u, v, w_i) in G .

For every candidate proposal a_i , we define:

$$W_i^+ = \sum \text{cumulative victory weights} \quad W_i^- = \sum \text{cumulative defeat weights}$$

The Weighted Copeland score is computed as:

$$\text{Score}(a_i) = W_i^+ - W_i^-$$

The global ordering is derived by sorting all proposals by decreasing score. Ties are resolved through a deterministic hierarchy prioritizing: (1) total win weight, (2) minimized loss count, (3) minimized residual risks, and (4) original configuration sequence.

Proposition 2 - Score Bounds

Statement. For any participating agent a_i , the Weighted Copeland score is bounded as follows:

$$-20 \leq \text{Score}(a_i) \leq 20$$

Proof. Following Theorem 2, each proposal is evaluated precisely twice. A proposal participates in exactly two comparisons as a candidate (not as a reviewer), and in each it is either winner or loser, contributing the weight to W_i^+ or W_i^- respectively. Given that each review assigns a weight in $[0, 10]$ to either W_i^+ or W_i^- , both sums are bounded within $[0, 20]$. Consequently, $\text{Score}(a_i)$ must reside within $[-20, 20]$. ■

4.4 Structural Invariants

The properties below follow directly from the deterministic ring assignment logic and ensure operational stability.

Theorem 4 - Self-Review Avoidance

Statement. For $N \geq 3$, no agent a_i evaluates its own proposal S_i .

Proof. According to the deterministic CDPC assignment protocol, any given reviewer a_i is tasked with evaluating the proposals generated by peer agents $a_{(i+1) \bmod N}$ and $a_{(i+2) \bmod N}$ exclusively. The occurrence of a self-review would necessitate that at least one of the following modular congruences be satisfied:

$$i + 1 \equiv i \pmod{N} \quad \text{or} \quad i + 2 \equiv i \pmod{N}$$

The former condition reduces to $1 \equiv 0 \pmod{N}$, which implies $N \mid 1$. This condition is satisfied only when $N = 1$, a case specifically omitted by the protocol constraints.

The latter condition simplifies to $2 \equiv 0 \pmod{N}$, which implies $N \mid 2$. This holds true only for populations where $N \in \{1, 2\}$, both of which are precluded by the formal requirement that $N \geq 3$.

Consequently, as neither congruence can be maintained, no agent is permitted to evaluate its own synthesized solution. ■

Theorem 5 - Ranking Determinism

Statement. The global ordering generated through the CDPC protocol is a purely deterministic function derived from the agent population, their respective proposals, and the resulting confrontation outcomes. Formally, we define the ranking function as:

$$R: (S_0, \dots, S_{N-1}, C_0, \dots, C_{N-1}) \rightarrow \pi$$

where $C_i = (u_i, v_i, w_i)$ represents the specific comparison outcome recorded by reviewer a_i , and π constitutes a total ordering of the population A . It follows that:

$$R(S_0, \dots, S_{N-1}, C_0, \dots, C_{N-1}) = R(S'_0, \dots, S'_{N-1}, C'_0, \dots, C'_{N-1})$$

in any instance where $S_i = S'_i$ and $C_i = C'_i$ holds for all $i \in \{0, \dots, N-1\}$.

Proof. By tracing the constituent computational phases of R , we demonstrate that each stage is inherently deterministic.

Step 1 - Reviewer assignment. The mapping of reviewer a_i to peer proposals $S_{(i+1) \bmod N}$ and $S_{(i+2) \bmod N}$ is governed by a closed-form arithmetic operation on i and N . This process is independent of stochastic variables or external states. For a fixed population N and constant agent sequence, the resulting assignment remains invariant across successive executions.

Step 2 - Edge set construction. Every comparison $C_i = (u_i, v_i, w_i)$ instantiates exactly one directed weighted edge (u_i, v_i, w_i) within the comparison graph G . Identical outcomes $C_i = C_i'$ necessarily yield identical edge sets E and E' , with the weight function w being pointwise equivalent to w' .

Step 3 - Score computation. For each participating agent a_i , the Weighted Copeland scores are defined as:

$$W_i^+ = \sum \text{cumulative victory weights} \quad W_i^- = \sum \text{cumulative defeat weights}$$

$$\text{Score}(a_i) = W_i^+ - W_i^-$$

These calculations represent finite sums over static edge sets and fixed weights. Given equivalent graphs, the scores $\text{Score}(a_i)$ must be identical to $\text{Score}'(a_i)$ for the entire population.

Step 4 - Sorting and tie-breaking. The final global ordering π is established by sorting agents according to their decreasing Weighted Copeland scores. Potential ties are resolved via a deterministic hierarchy applied in the following order:

1. Cumulative victory weight W_i^+ (higher weight prevails)
2. Total count of losses (lower count prevails)
3. Reported residual risks within the proposal (fewer risks prevail)
4. Original configuration sequence (lower index prevails)

Each tie-breaking criterion constitutes a deterministic function of the observed edge set, weights, proposals, and indices. No external or random variables are introduced. Given identical inputs, the resolved ordering π is necessarily identical to π' .

Since every constituent phase of R is a deterministic function, the ranking function R itself is globally deterministic. ■

Corollary - End-to-End Reproducibility.

Any two executions of the HALc protocol that receive equivalent proposals and generate identical comparison outcomes will yield identical global rankings. This confirms that non-determinism within HALc originates exclusively from underlying LLM inference rather than the coordination framework itself.

4.5 Computational Complexity

The cumulative LLM interaction overhead is decomposed as follows:

- **Synthesis stage.** Each of the N agents generates a single proposal: N total calls.
- **Comparison stage.** Each of the N agents performs exactly one confrontation: N total calls.
- **Ledger generation.** One optional summarizer interaction: $\delta \in \{0, 1\}$.

Total Interaction Count: $T(N) = 2N + \delta$, concluding $T(N) \in O(N)$.

The deterministic local computational overhead is detailed below:

Operation	Complexity
Markdown semantic extraction	$O(L)$, L = cumulative output length
Comparison graph construction	$O(N)$
Copeland scoring engine	$O(N)$
Deterministic global sort	$O(N \log N)$
Tiered tie-breaking logic	$O(N)$

Cumulative local overhead is calculated at $O(N \log N)$. This computational cost is negligible when contrasted with the inherent latency of LLM inference.

4.6 Summary of Formal Guarantees

Property	Guarantee
Comparisons per reviewer	Exactly 1
Evaluations per proposal	Exactly 2
Avoidance of self-review	Absolute for $N \geq 3$
Single connected comparison component	Guaranteed for $N \geq 3$
Weighted score interval	$[-20, 20]$
Execution determinism	Guaranteed via fixed assignments
Inference complexity	$O(N)$
Aggregator complexity	$O(N \log N)$
Debate model complexity	$O(N^2)$

4.7 Discussion - The Approximation Gap

The gap between the sparse CDPC ranking and the exhaustive ranking has three practical consequences that HALc users and future researchers should consider.

1. The ranking is evidence-based but not complete. Every ordering produced by HALc is grounded in real comparison evidence across a connected graph. No pair is ranked by tie-breaking alone. But transitively inferred orderings carry less evidentiary weight than directly observed ones. HALc's ranking should be interpreted as a well-grounded approximation rather than a ground-truth preference order.

2. Confidence weights partially compensate. Strongly expressed preferences propagate more weight through the graph than weakly expressed ones. When reviewer agents are well-calibrated, high-confidence comparisons tend to dominate the ranking, reducing the

influence of transitively inferred orderings. This is one reason why confidence calibration, discussed in Section 7.3, is operationally important.

3. Formal approximation bounds remain an open problem. This paper does not establish a theoretical bound on the distance between the CDPC ranking and the exhaustive ranking — for example, in terms of Kendall tau distance or Spearman rank correlation. Deriving such bounds, potentially as a function of N , graph density, and confidence calibration quality, would substantially strengthen the theoretical foundations of HALc and is identified as a priority direction for future work in Section 7.11.

Corollary — Practical Implication for Enterprise Deployments

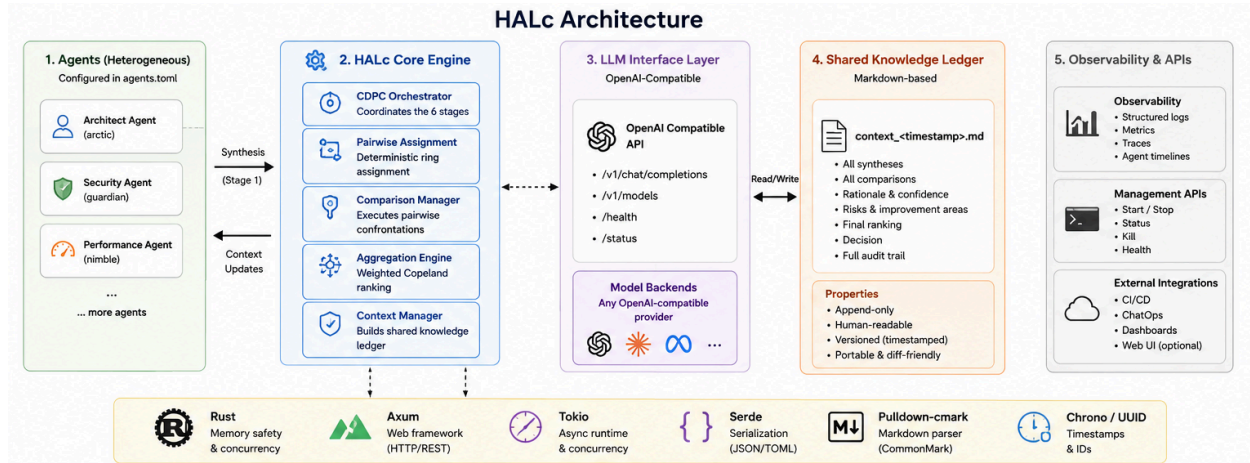
For enterprise use cases where the relative ordering of the top-ranked proposals is the primary decision signal, the approximation gap is likely acceptable: the highest-scoring proposal under CDPC has won both of its observed comparisons with positive confidence, making it a strong candidate regardless of unobserved pairings. For use cases requiring exhaustive verification of relative ordering across all proposals, a denser comparison topology should be considered.

5. Implementation

HALc is implemented as a native Rust application that exposes an OpenAI-compatible HTTP API while orchestrating a heterogeneous population of Large Language Model (LLM) agents. The implementation separates reasoning from coordination: agents perform reasoning, whereas HALc performs orchestration, parsing, ranking, persistence, and execution tracing. This separation allows any OpenAI-compatible endpoint to participate without modifying the coordination algorithm.

5.1 Architectural Overview

HALc is composed of six principal subsystems: (1) an OpenAI-compatible REST API implemented with Axum, (2) an orchestration engine implementing Compact Distributed Pairwise Confrontation (CDPC), (3) an agent communication layer, (4) a Markdown parser and semantic extractor, (5) a ledger responsible for traceability and context evolution, and (6) a configuration subsystem. Each subsystem is independent, enabling future replacement or extension without affecting the protocol.



5.2 Technology Stack

The implementation is written in Rust to provide memory safety, deterministic behavior and efficient concurrency. `Axum` is used for the HTTP server, `Tokio` for asynchronous execution, `Serde` for configuration parsing, `Request` for outbound OpenAI-compatible requests, and tracing for structured logging. Agent responses remain Markdown-first; HALC never requires JSON-formatted reasoning from participating models.

5.3 Agent Configuration

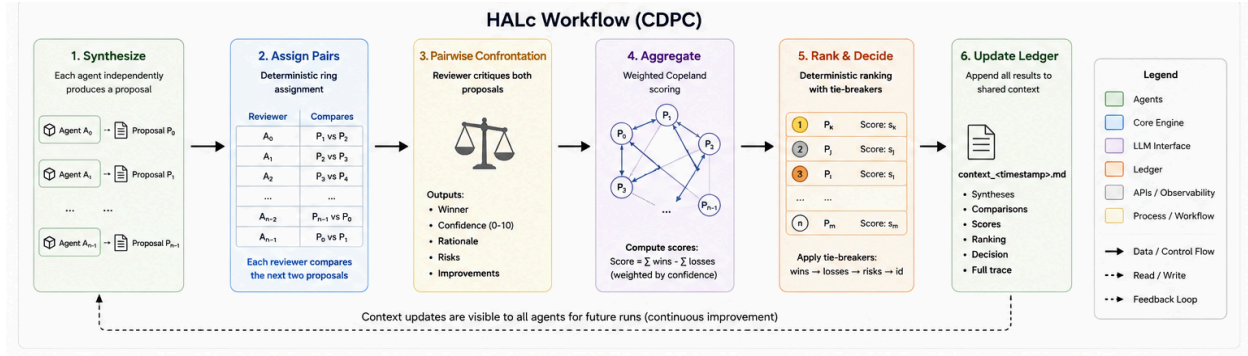
Agents are declared in `agents.toml`. Each entry defines the endpoint URL, model identifier, API key, system prompt, inference parameters, enabled state, and execution role. Agents may use different providers and models simultaneously. A global configuration defines the confrontation mode and the maximum response length accepted from every LLM.

5.4 OpenAI-Compatible Interface

HALC exposes the standard `/v1/chat/completions` endpoint, allowing existing OpenAI clients to communicate with the framework without modification. Incoming requests are transformed into an internal confrontation request. The final ranked answer is returned as a standard assistant response, preserving compatibility with existing SDKs and applications.

5.5 Orchestration Engine

The orchestration engine implements the six stages of CDPC: agent synthesis, deterministic pairwise assignment, pairwise comparison, Weighted Copeland aggregation, final decision generation, and shared context evolution. Agent synthesis and comparisons are executed concurrently whenever possible. The remaining stages are deterministic local computations.



5.6 Markdown Communication Protocol

All communication between HALc and participating agents is performed using Markdown. Agents receive structured prompts and return structured Markdown documents containing headings such as Problem Statement, Proposal, Risks and Evidence. HALc extracts semantic sections using heading normalization rather than relying on brittle JSON schemas. This approach improves interoperability across heterogeneous LLM providers while preserving natural language reasoning.

5.7 Pairwise Assignment

Reviewer assignments are generated deterministically from the order of agents in the configuration. Every reviewer evaluates exactly two peer proposals and never evaluates its own. This deterministic topology guarantees reproducibility across executions given identical inputs.

5.8 Weighted Copeland Aggregation

Each pairwise comparison produces a winner, a loser and a confidence value. HALc constructs a weighted comparison graph and computes a Weighted Copeland score for every proposal. The framework then produces a complete ranking using deterministic tie-breaking rules based on win weight, losses, residual risks and configuration order.

5.9 Shared Knowledge Ledger

HALc maintains a versioned ledger containing execution metadata, confrontation events, rankings and generated contexts. After each execution a new Markdown document named `context_<timestamp>.md` is written to the ledger directory. This document contains the user request, winning proposal, complete ranking, lessons learned, remaining risks and unresolved disagreements. The generated document becomes the shared context for the next execution, enabling continuous organizational learning.

5.10 Observability and Traceability

Every execution stage emits structured events to the console and ledger. Normal mode presents hierarchical progress, verbose mode adds timings, and debug mode records prompts and responses while redacting credentials. This design provides reproducibility and simplifies protocol analysis and benchmarking.

5.11 Error Handling

HALc tolerates partial failures. Invalid Markdown sections, oversized responses, unavailable endpoints and failed comparisons are isolated whenever possible. Failed comparisons are excluded from the ranking while the remaining valid comparisons continue the confrontation. This allows the framework to operate in heterogeneous and unreliable environments.

5.12 Testing Strategy

The implementation is validated through unit, integration and property-based tests. Unit tests verify assignment generation, Markdown parsing, ranking, tie-breaking and configuration parsing. Integration tests use mocked OpenAI-compatible endpoints to validate end-to-end executions. Property tests verify invariants such as deterministic assignments, absence of self-review, connected comparison graphs and stable rankings under deterministic inputs.

5.13 Extensibility

HALc deliberately separates the coordination framework from the coordination algorithm. Although the current implementation uses Compact Distributed Pairwise Confrontation, the architecture permits future algorithms such as hierarchical confrontation, adaptive reviewer assignment, Bayesian confidence estimation or tournament-based aggregation without changing the public API or the ledger model.

5.14 Experimental Design

Note: While empirical findings are pending, this section delineates the rigorous evaluation methodology intended for subsequent execution.

5.14.1 Research Questions

- **RQ1 (Quality Analysis).** Can HALc deliver reasoning excellence comparable to all-to-all debate while minimizing communication overhead?
- **RQ2 (Scaling Behavior).** Does actual execution latency and resource consumption mirror the projected linear scaling with N?
- **RQ3 (Ledger Utility).** To what extent does the versioned knowledge ledger enhance decision fidelity across iterative confrontation sessions?

5.14.2 Baselines

Baseline	Description
Standalone Expert	Optimal individual agent performance without coordination.
Exhaustive Debate	Full N-to-N connectivity, multi-round critique at $O(N^2)$ cost.
LLM-Blender	Ensembling via external cross-encoder ranking (Jiang et al., 2023).
AgentPrune	Learned, task-adaptive sparse communication (Zhang et al., 2025a).
HALc (Isolated)	Protocol execution with semantic context reset for each trial.

5.14.3 Benchmark Tasks

Evaluation spans three enterprise-grade reasoning categories:

Architecture Synthesis. Agents analyze system requirements to propose and critique architectural frameworks. Ground truth derived from expert committee consensus.

Cybersecurity Auditing. Agents identify and prioritize vulnerabilities within infrastructure specifications, validated against CVE-indexed reference sets.

Strategic Decisioning. Agents formulate prioritized action plans for business scenarios, evaluated through double-blind human preference studies.

5.14.4 Metrics

Metric	Target RQ
Win-rate comparison	RQ1
Token-per-decision efficiency	RQ1, RQ2
Wall-clock scaling vs N	RQ2

Fidelity delta over sessions	RQ3
Subjective expert preference	RQ1, RQ3

5.14.5 Ledger Accumulation Protocol

To isolate the specific efficacy of the shared versioned ledger, benchmarks are conducted under two divergent conditions:

- **Cold-Start HALc:** Execution occurs with a blank semantic context for each trial.
- **Warm-Start HALc:** Execution incorporates the full evolutionary ledger from all preceding confrontations.

The fidelity improvement across sequential sessions serves as a quantifiable metric for collective organizational learning - representing the core contribution of this research. This longitudinal analysis is expected to highlight the protocol's unique value proposition independently of baseline comparisons.

5.14.6 Agent Population

Experiments are staged with a diverse agent population ($N \in \{3, 5, 7, 10\}$) sourced from multiple OpenAI-compatible providers. Agents are differentiated by distinct system prompts reflecting specialized domain expertise, thereby validating the protocol's provider-agnostic interoperability and scaling consistency.

6. Discussion

The Compact Distributed Pairwise Confrontation protocol represents a deliberate design choice rather than a claim of theoretical superiority. The academic literature has made substantial progress on communication efficiency, sparse topology learning, and pairwise aggregation. HALc does not attempt to advance those fronts. Instead, it asks a different question: given what the research community already knows about multi-agent coordination, what does a deployable enterprise system actually look like?

The answers that HALc proposes are architectural and operational rather than algorithmic.

6.1 Coordination Rather than Debate

Most existing multi-agent reasoning frameworks treat additional interaction as inherently beneficial. Debate-based systems encourage every participant to critique every competing proposal, often through multiple rounds. While this maximizes information exchange, it also

maximizes computational cost, latency, and token consumption. As the number of participating agents grows, communication rapidly becomes the dominant cost.

HALc adopts the principle established by Chen et al. (2024) and extended by AgentPrune, AgentDropout, and G-Designer: that sparse communication can preserve most of the quality benefit of exhaustive debate at a fraction of the cost. Where HALc differs from those systems is in the nature of the sparsity. Rather than learning an optimal sparse topology for each task, HALc uses a fixed deterministic ring that is identical across executions. This sacrifices task-specific optimality in favor of operational predictability - a trade-off that is rarely justified in academic benchmarks but frequently required in enterprise deployments.

6.2 Separation of Reasoning and Coordination

A central architectural decision in HALc is the strict separation between reasoning and coordination. Reasoning remains entirely within the participating LLMs. Each agent receives the same constitutional context but applies its own prompt, model, and inference parameters. HALc itself never reasons about the problem domain.

Instead, HALc performs deterministic orchestration: scheduling, pairwise assignment, Markdown parsing, Weighted Copeland ranking, ledger generation, and execution tracing. This separation has two practical consequences. First, it enables interoperability across any OpenAI-compatible endpoint without requiring identical APIs, output schemas, or provider-specific capabilities. Second, it ensures that identical agent outputs always produce identical rankings, because every coordination step is fully deterministic.

This property is largely absent from adaptive topology systems, which introduce stochastic or learned components into the coordination layer itself. For enterprise environments where audit requirements demand reproducible decisions, deterministic coordination is not a limitation but a design goal.

6.3 Markdown as a Semantic Exchange Format

Many existing orchestration frameworks require agents to exchange rigid JSON structures. While suitable for deterministic APIs, this constrains reasoning and typically requires provider-specific prompt engineering to elicit reliably parseable output.

HALc instead adopts Markdown as the native communication format, following the observation of Jiang et al. (2023) that natural language comparison produces higher-quality pairwise distinctions than structured scoring alone. Structured headings provide sufficient semantic organization for deterministic section extraction while allowing agents to express complex reasoning naturally. This improves interoperability across heterogeneous LLM providers and eliminates dependence on provider-specific structured output capabilities.

Markdown also provides long-term human readability. Every synthesis, comparison, and context document can be inspected directly by human reviewers without additional tooling, which directly supports the governance and traceability requirements common in regulated enterprise environments.

6.4 The Shared Knowledge Ledger as Primary Contribution

Unlike the pairwise confrontation mechanism and the Weighted Copeland aggregation - both of which have clear precedents in the literature - the shared versioned knowledge ledger represents HALc's most distinctive architectural contribution.

Existing multi-agent systems either maintain isolated memories for individual agents or discard collective knowledge entirely between sessions. Neither approach is suitable for enterprise deployments, where decisions accumulate over time and organizational context must be preserved and auditable.

HALc addresses this through a simple but consequential design: after every confrontation, a new versioned Markdown document is generated capturing the user request, winning proposal, complete ranking, lessons learned, remaining risks, and unresolved disagreements. This document becomes the shared constitutional context for all agents in the next execution.

This design has three operational consequences. First, organizational knowledge evolves collectively rather than individually, avoiding the fragmentation that arises when agents accumulate divergent private memories. Second, the ledger provides a complete and human-readable audit trail of collective decision making over time. Third, the append-only versioned structure means that previous contexts remain immutable and can be retrieved to understand how collective knowledge evolved across executions.

No comparable mechanism appears in the multi-agent coordination literature reviewed in Section 2, making the shared versioned ledger the contribution most likely to offer genuine value to practitioners and researchers working on long-running enterprise AI systems.

6.5 Continuous Organizational Learning

Unlike traditional conversational systems, HALc does not maintain isolated memories for individual agents. Instead, every confrontation generates a new versioned context shared by the entire population.

This design has some consequences.

First, organizational knowledge evolves collectively rather than individually. Lessons learned during one confrontation immediately become available to all agents in subsequent executions.

Second, the shared ledger provides a complete audit trail of decision making. Previous contexts remain immutable and can be revisited to understand how collective knowledge evolved over time.

Finally, maintaining a common semantic context reduces the risk of knowledge fragmentation, where individual agents gradually diverge because they accumulate different experiences.

6.6 Confidence-Weighted Decision Making

The use of Weighted Copeland aggregation introduces an explicit distinction between preference and confidence. Rather than recording only which proposal is preferred, reviewers also express how strongly they support that preference.

This additional information enables stronger consensus to have greater influence on the final ranking while still preserving minority opinions. The resulting ranking therefore reflects both the direction and the intensity of collective preferences.

Although other social choice mechanisms could be employed, Weighted Copeland offers a practical balance between computational efficiency, interpretability, and compatibility with sparse comparison graphs.

6.7 Scalability Considerations

The principal computational advantage of HALc relative to exhaustive debate grows with the number of participating agents. Traditional all-to-all debate requires $O(N^2)$ LLM interactions for N agents. HALc requires exactly $2N$ interactions plus one optional context generation call, achieving linear scaling while maintaining connected comparison coverage and deterministic aggregation.

Relative to adaptive topology systems such as AgentPrune and G-Designer, HALc does not claim superior reasoning quality. Those systems optimize topology for task performance and may outperform HALc on specific benchmarks. The relevant comparison for enterprise deployments is different: HALc offers linear cost, deterministic execution, zero topology training overhead, and immediate deployability across heterogeneous providers - properties that adaptive systems do not currently provide.

6.8 Applicability to Enterprise AI

Enterprise AI systems increasingly consist of heterogeneous agents specialized in different domains: software architecture, cybersecurity, regulatory compliance, infrastructure operations, legal analysis, and product management. In these environments, coordination often becomes more difficult than reasoning itself. Different models expose different APIs, possess different capabilities, and evolve independently over time.

HALc addresses this through a provider-independent coordination layer. Because communication is based on OpenAI-compatible endpoints and Markdown documents, new agents can be incorporated without modifying the protocol. This architecture also directly supports the governance requirements common in enterprise deployments: full traceability through structured logging, reproducibility through deterministic execution, explainability through human-readable Markdown documents, and organizational memory through the versioned knowledge ledger.

6.9 Trade-offs

The design choices in HALc involve explicit trade-offs that should be stated clearly.

Because every proposal is evaluated exactly twice, HALc intentionally sacrifices exhaustive peer review. In scenarios where every proposal must be examined by every expert, traditional debate or denser topologies will identify issues that HALc's sparse comparison does not capture.

The fixed ring assignment does not account for domain expertise. A security-oriented agent may review an infrastructure proposal outside its specialization. Adaptive systems that match reviewers to proposals by semantic similarity will outperform HALc on tasks where domain alignment matters.

The quality of the final ranking depends on the calibration of confidence values produced by reviewer agents. Modern language models are not consistently calibrated estimators, and different models may interpret the confidence scale differently across a heterogeneous population.

Finally, the shared ledger will grow over time, increasing inference costs and consuming context window capacity. Production deployments will require context management strategies such as summarization, semantic compression, or retrieval-augmented generation.

These trade-offs are not defects but consequences of the enterprise design priorities that HALc optimizes for. Systems that prioritize reasoning quality over operational simplicity should consider adaptive topology approaches. HALc is designed for environments where reproducibility, auditability, and provider independence take precedence.

6.10 Relationship to Existing Work

HALc does not attempt to replace existing multi-agent reasoning frameworks. Instead, it combines ideas originating from Constitutional AI, collaborative reasoning, constructive critique, software architecture reviews, and social choice theory into a unified coordination protocol.

From Constitutional AI, HALc adopts the concept of an explicit constitutional context that guides decision making.

From collaborative reasoning, it preserves independent proposal generation while substantially reducing communication.

From constructive architecture reviews, it adopts structured confrontation rather than unstructured debate.

From social choice theory, it adopts confidence-weighted pairwise aggregation to transform distributed comparisons into a deterministic collective ranking.

The resulting framework therefore occupies a distinct position within the landscape of multi-agent AI systems: it focuses on scalable coordination rather than maximizing conversational interaction.

6.11 Summary

HALc demonstrates that effective multi-agent coordination does not necessarily require exhaustive communication. By combining independent reasoning, deterministic sparse confrontation, confidence-weighted aggregation, and a shared evolving knowledge ledger, the framework provides a scalable and explainable coordination protocol for heterogeneous LLM populations.

The central hypothesis of this work is that, for many enterprise reasoning tasks, carefully structured interactions can preserve most of the benefits of collaborative intelligence while significantly reducing computational cost. The experimental evaluation presented in this paper is intended to test this hypothesis and to quantify the practical trade-offs between communication efficiency, reasoning quality, and scalability.

7. Limitations

HALc introduces a coordination protocol whose primary value is architectural and systems-oriented rather than theoretical. The limitations below should be interpreted in that context.

7.1 Absence of Empirical Evaluation

The most significant limitation of the current work is the absence of empirical evaluation against comparable baselines. The mathematical analysis establishes desirable properties - linear communication complexity, deterministic execution, connected comparison graphs - but does not demonstrate that HALc produces competitive reasoning quality relative to exhaustive debate, learned sparse topologies, or LLM-Blender-style ensembling.

Future work should evaluate HALc on standardized benchmarks covering software engineering, architectural decision making, mathematical reasoning, and enterprise planning tasks,

comparing against Du et al. (2024), Chen et al. (2024), AgentPrune (Zhang et al., 2025a), and LLM-Blender (Jiang et al., 2023) under equivalent agent populations and token budgets.

This evaluation is essential to establish where in the quality-cost trade-off space HALc actually sits, and under what conditions the enterprise design priorities it optimizes for justify any reasoning quality reduction relative to more exhaustive approaches.

7.2 Dependence on LLM Quality

HALc assumes that participating agents are capable of producing coherent proposals and meaningful peer reviews. The quality of the collective decision therefore remains bounded by the quality of the underlying language models.

Poorly configured agents, inaccurate models, or inadequate prompts may reduce the quality of the final ranking regardless of the coordination algorithm itself.

HALc improves coordination but does not compensate for fundamentally incorrect reasoning produced by individual agents.

7.3 Confidence Calibration

The current protocol relies on reviewer agents to assign confidence values to pairwise comparisons. These confidence values directly influence the Weighted Copeland ranking.

However, modern language models are not consistently calibrated estimators of confidence. Different models may interpret the confidence scale differently, leading to inconsistent weighting across heterogeneous agent populations.

Future versions of HALc should investigate confidence normalization, probabilistic calibration, or historical reviewer reliability to improve the robustness of confidence-weighted aggregation.

7.4 Sparse Comparison Topology

The Compact Distributed Pairwise Confrontation protocol deliberately minimizes the number of comparisons performed during each confrontation.

While this reduction provides substantial computational savings, it also limits the amount of information available for ranking. Every proposal is evaluated exactly twice rather than by every other participant.

Consequently, subtle differences between closely related proposals may remain undetected. In applications where exhaustive peer review is essential, a denser comparison topology may provide higher confidence at the expense of additional computation.

7.5 Static Reviewer Assignment

The current implementation uses a deterministic ring assignment to guarantee balanced reviewer workloads and reproducible execution.

Although mathematically attractive, this strategy does not consider domain expertise. A security-oriented agent may review infrastructure proposals, while a software architecture specialist may compare legal analyses. Adaptive reviewer assignment based on expertise or semantic relevance - as explored in AgentPrune and G-Designer - would likely improve ranking quality at the cost of determinism.

Future versions of HALc should investigate adaptive reviewer assignment based on expertise, historical performance, or semantic similarity between proposals.

7.6 Shared Context Growth

HALc continuously evolves a shared Markdown context after every confrontation. Although this mechanism enables collective organizational learning, the context will inevitably grow over time.

Large contexts increase inference costs, consume additional context window capacity, and may introduce outdated or redundant information.

Practical deployments will therefore require context management strategies such as summarization, semantic compression, retrieval-augmented generation, or hierarchical knowledge organization.

7.7 Single Shared Memory

HALc intentionally replaces independent agent memories with a single shared organizational context.

While this design prevents knowledge fragmentation, it also reduces diversity in long-term learning. Agents progressively converge toward the same shared knowledge base, potentially reducing independent perspectives over extended execution histories.

Future research may investigate hybrid memory models combining a shared organizational ledger with limited agent-specific memories.

7.8 Robustness to Faulty or Adversarial Agents

The current protocol assumes that participating agents act in good faith and produce honest evaluations.

HALc does not currently include mechanisms for detecting adversarial reviewers, biased confidence assignments, collusion between agents, or systematically unreliable participants.

In open or partially trusted environments, reputation systems, anomaly detection, Byzantine fault-tolerant aggregation, or reviewer reliability estimation may become necessary.

7.9 Determinism vs. Exploration

HALc intentionally favors deterministic execution to improve reproducibility and auditability.

While deterministic coordination simplifies experimentation and debugging, it may also reduce exploration. Randomized reviewer assignments or stochastic confrontation schedules could expose additional pairwise comparisons over repeated executions, potentially improving ranking quality.

Balancing determinism and exploration remains an open research question.

7.10 Generality of the Coordination Protocol

The protocol has been designed for heterogeneous LLM agents communicating through OpenAI-compatible APIs using Markdown documents.

Although these assumptions cover many current enterprise deployments, they may not directly apply to symbolic reasoning systems, reinforcement learning agents, or multimodal agents operating on images, audio, or structured knowledge graphs.

Extending HALc beyond text-centric language models represents an important direction for future research.

7.11 Absence of Formal Optimality Proof

This paper demonstrates key desirable mathematical properties of Compact Distributed Pairwise Confrontation, including linear communication complexity, deterministic execution, balanced reviewer workloads, and connected comparison graphs.

However, it does not formally prove that the deterministic ring assignment is the optimal sparse comparison topology under all possible constraints. Likewise, the paper does not establish theoretical approximation bounds relating the Weighted Copeland ranking obtained from sparse comparisons to the ranking that would result from exhaustive all-to-all confrontation.

Developing such optimality and approximation proofs would strengthen the theoretical foundations of HALc, and remains an important avenue for future work.

7.12 Summary

The limitations above define a clear research agenda. Empirical evaluation against established baselines is the most urgent priority and would substantially strengthen the claims made in this paper. Confidence calibration, adaptive reviewer assignment, and context management are the most practically relevant directions for production deployment. Formal optimality analysis and robustness to adversarial agents represent longer-term theoretical contributions.

HALc should be understood as a deployable baseline for enterprise multi-agent coordination - one that establishes a practical starting point for systems where operational constraints dominate, while leaving substantial room for refinement as empirical evidence accumulates.

References

Multi-Agent Reasoning and Constitutional AI

1. Bai, Y., Jones, A., Ndousse, K., Askell, A., Chen, A., DasSarma, N., ... Kaplan, J. (2022). Constitutional AI: Harmlessness from AI Feedback. Anthropic Technical Report. Retrieved from <https://www.anthropic.com/research/constitutional-ai-harmlessness-from-ai-feedback>
https://www-cdn.anthropic.com/7512771452629584566b6303311496c262da1006/Anthropic_ConstitutionalAI_v2.pdf
2. Meta AI. (2025). *Collaborative Reasoner: Self-Improving Social Agents with Synthetic Conversations*. <https://ai.meta.com/research/publications/collaborative-reasoner-self-improving-social-agents-with-synthetic-conversations/>
3. Chen, Y., et al. (2024). Improving Multi-Agent Debate with Sparse Communication Topology. Workshop on Advancing LLM-Based Multi-Agent Collaboration, AAAI 2025. Retrieved from https://multiagents.org/2025_talks/talk_improving_multi_agent_debate_with_sparse_communication_topology.pdf
4. Du, Y., Li, S., Torralba, A., Tenenbaum, J., & Mordatch, I. (2024). *Improving Factuality and Reasoning in Language Models through Multi-Agent Debate*. Proceedings of ICML.
5. Madaan, A., et al. (2024). *Self-Refine: Iterative Refinement with Self-Feedback*. NeurIPS.
6. Shinn, N., et al. (2024). *Reflexion: Language Agents with Verbal Reinforcement Learning*. NeurIPS.
7. Zhang, G., Yue, Y., Li, Z., Yun, S., Wan, G., Wang, K., Cheng, D., Yu, J.X., & Chen, T. (2025a). Cut the Crap: An Economical Communication Pipeline for LLM-based Multi-Agent Systems. *The Thirteenth International Conference on Learning Representations (ICLR)*. <https://arxiv.org/abs/2410.02506>
8. Wang, Z., Wang, Y., Liu, X., Ding, L., Zhang, M., Liu, J., & Zhang, M. (2025b). AgentDropout: Dynamic Agent Elimination for Token-Efficient and High-Performance LLM-Based Multi-Agent Collaboration. *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*. <https://arxiv.org/abs/2503.18891>
9. Zhang, G., Yue, Y., Sun, X., Wan, G., Yu, M., Fang, J., Wang, K., Chen, T., & Cheng, D. (2025b). G-Designer: Architecting Multi-agent Communication Topologies via Graph Neural Networks. *Forty-second International Conference on Machine Learning (ICML)*. <https://arxiv.org/abs/2410.11782>
10. Jiang, D., Ren, X., & Lin, B. Y. (2023). LLM-Blender: Ensembling Large Language Models with Pairwise Ranking and Generative Fusion. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL 2023)* (pp. 14165–14178). Association for Computational Linguistics. <https://doi.org/10.18653/v1/2023.acl-long.792>
11. Kaesberg, L. B., Becker, J., Wahle, J. P., Ruas, T., & Gipp, B. (2025). Voting or Consensus? Decision-Making in Multi-Agent Debate. In *Findings of the Association for*

Computational Linguistics: ACL 2025. University of Göttingen. Retrieved from <https://aclanthology.org/2025.findings-acl.606.pdf>

12. Lanctot, M., et al. (2025). Voting as Evaluation: Copeland, Ranked Pairs, and Iterative Maximal Lotteries for LLM Agent Ranking. In *Evaluating Generalization Capabilities of LLM-Based Agents Using Concordia*. *arXiv preprint arXiv:2512.03318*. Retrieved from <https://arxiv.org/html/2512.03318v1>

Agent Memory Systems

13. Packer, C., Fang, V., Patil, S. G., Lin, K., Wooders, S., & Gonzalez, J. E. (2023). MemGPT: Towards LLMs as Operating Systems. *arXiv preprint arXiv:2310.08560*.
14. Xu, W., Liang, Z., Mei, K., Gao, H., Tan, J., & Zhang, Y. (2025). A-MEM: Agentic Memory for LLM Agents. *arXiv preprint arXiv:2502.12110*.
15. Rasmussen, P., Paliychuk, P., Beauvais, T., Ryan, J., & Chalef, D. (2025). Zep: A Temporal Knowledge Graph Architecture for Agent Memory. *arXiv preprint arXiv:2501.13956*.
16. Chase, H. (2024). LangMem. LangChain. <https://github.com/langchain-ai/langmem>.

Enterprise AI Coordination

17. Microsoft. (2025). *The Future of Agentic Computing*.
18. TechTarget. (2025). *Enterprise AI is Becoming a Coordination Problem*.
19. Computer Weekly. (2025). *What is Causing Agentic AI's Enterprise Gap and How to Fix It?*
20. The Verge. (2025). *Microsoft's Claude Code Integration and the Evolution of AI Coding Assistants*.

Software Architecture and Constructive Confrontation

21. Rational Software Corporation. *Rational Unified Process – Software Architecture Review*. https://files.defcon.no/RUP/process/workers/wk_archt.htm
 22. Bass, L., Clements, P., & Kazman, R. (2022). *Software Architecture in Practice* (4th ed.). Addison-Wesley.
 23. Rozanski, N., & Woods, E. (2019). *Software Systems Architecture* (3rd ed.). Addison-Wesley.
 24. Kruchten, P. (1995). *Architectural Blueprints-The 4+1 View Model of Software Architecture*. IEEE Software.
-

Distributed Systems

25. Li, Z., et al. (2025). ARG-Designer: Autoregressive Graph Generation for Multi-Agent Communication Topology. *arXiv preprint*. Retrieved from <https://arxiv.org/pdf/2603.19677>
 26. Tran, A., et al. (2026). GoAgent: Group-of-Agents Communication Topology Generation for LLM-Based Multi-Agent Systems. *arXiv preprint arXiv:2603.19677*. Retrieved from <https://arxiv.org/pdf/2603.19677>
 27. Du, Y., Li, S., Torralba, A., Tenenbaum, J., & Mordatch, I. (2024). Improving Factuality and Reasoning in Language Models through Multi-Agent Debate. In *Proceedings of the International Conference on Machine Learning (ICML 2024)*.
 28. Ongaro, D., & Ousterhout, J. (2014). *In Search of an Understandable Consensus Algorithm (Raft)*. USENIX ATC.
 29. Lamport, L. (1978). *Time, Clocks, and the Ordering of Events in a Distributed System*. Communications of the ACM.
 30. Lamport, L. (1998). *The Part-Time Parliament*. ACM Transactions on Computer Systems.
 31. Chandra, T., & Toueg, S. (1996). *Unreliable Failure Detectors for Reliable Distributed Systems*. Journal of the ACM.
 32. Birman, K. (2012). *Guide to Reliable Distributed Systems*. Springer.
-

Social Choice Theory

33. Copeland, A. H. (1951). *A Reasonable Social Welfare Function*.
 34. Condorcet, M. (1785). *Essay on the Application of Analysis to the Probability of Majority Decisions*.
 35. Kemeny, J. G. (1959). *Mathematics without Numbers*.
 36. Young, H. P. (1988). *Condorcet's Theory of Voting*. American Political Science Review.
 37. Schulze, M. (2011). *A New Monotonic and Clone-Independent Single-Winner Election Method*. Voting Matters.
 38. Tideman, T. N. (1987). *Independence of Clones as a Criterion for Voting Rules*. Social Choice and Welfare.
 39. Saari, D. G. (1995). *Basic Geometry of Voting*. Springer.
-

Decision Theory and Collective Intelligence

40. Surowiecki, J. (2004). *The Wisdom of Crowds*. Doubleday.
41. Wooldridge, M. (2009). *An Introduction to MultiAgent Systems* (2nd ed.). Wiley.
42. Russell, S., & Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson.

- 43. Malone, T. W. (2018). *Superminds: The Surprising Power of People and Computers Thinking Together*. Little, Brown.
 - 44. Klein, G. (2007). *Naturalistic Decision Making*. Human Factors.
-

Large Language Models

- 45. Brown, T., et al. (2020). *Language Models are Few-Shot Learners*. NeurIPS.
 - 46. OpenAI. (2023). *GPT-4 Technical Report*.
 - 47. Anthropic. (2024). *Claude 3 Model Card*.
 - 48. Gemini Team. (2024). *Gemini Technical Report*.
 - 49. DeepSeek AI. (2024). *DeepSeek LLM Technical Report*.
 - 50. Touvron, H., et al. (2023). *Llama 2: Open Foundation and Fine-Tuned Chat Models*.
-

Mathematical Foundations

- 51. Diestel, R. (2017). *Graph Theory* (5th ed.). Springer.
 - 52. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4th ed.). MIT Press.
 - 53. Knuth, D. E. (1997). *The Art of Computer Programming, Volume 1: Fundamental Algorithms*. Addison-Wesley.
 - 54. Papadimitriou, C. H. (1994). *Computational Complexity*. Addison-Wesley.
 - 55. Sipser, M. (2012). *Introduction to the Theory of Computation* (3rd ed.). Cengage.
-

HALc

- 56. Gegout, C. (2026). *HALc: Heuristic Agent Ledger by Confrontation - A Distributed Coordination Protocol for Heterogeneous Large Language Model Agents*. (This paper available at <https://github.com/gegout/HALc>)